

Certificate Generation Guide

In order to generate a self-signed certificate you need the following elements:

- **CA Private Key** - represents a private key of a fake Certificate Authority, used to sign the CA Certificate
- **CA Certificate** - represents a certificate of a fake Certificate Authority, used to sign Client's Certificate Signature Request (CSR)
- **Client Private Key** - represents a private key of a fake TPP certificate.

In order to successfully generate all this elements and sign a fake TPP certificate, it is required to follow this steps:

1. Generate CA and Client private keys:

```
1 openssl genrsa -out ca_private.key 2048
2 openssl genrsa -out client_private.key 2048
```

2. Generate openssl configuration for CA CSR:

configuration should be included in a file called, for example, `ca_openssl.cnf` with the following content:

```
1 oid_section = my_oid
2
3 [ req ]
4 default_bits = 2048
5 prompt = no
6 encrypt_key = no
7 default_md = sha1
8 distinguished_name = dn
9
10 [ dn ]
11 CN = Fake CA Authority
12 O = Fake CA
13 C = UK
14 ST = Fake street
15
16 [ cert_ext ]
17 subjectKeyIdentifier=hash
18 keyUsage=critical,digitalSignature,keyEncipherment
19 extendedKeyUsage=clientAuth,serverAuth
20
21 [ my_oid ]
22 organizationIdentifier=2.5.4.97
```

3. Create CA CSR:

```
1 openssl req -config ca_openssl.cnf -new -key ca_private.key -nodes -out ca.csr
```

4. Create CA self signed certificate from CA CSR

```
1 openssl x509 -signkey ca_private.key -in ca.csr -req -days 365 -out ca_certificate.crt
```

5. Generate openssl configuration for Client CSR:

configuration should be included in a file called, for example, `client_openssl.cnf` with the following structure.

```
1 oid_section = OIDs
2
3 [ req ]
4 default_bits = 2048
5 prompt = no
6 encrypt_key = no
7 default_md = sha1
8 distinguished_name = dn
9 req_extensions = cert_ext
10
11 [ dn ]
12 CN = Fake TPP
13 O = Fake TPP
14 C = UK
15 ST = Fake Street
16 organizationIdentifier = PGB-123
17
18 [ cert_ext ]
19 basicConstraints = CA:TRUE
20 subjectKeyIdentifier = hash
21 keyUsage = critical,digitalSignature,keyEncipherment
22 extendedKeyUsage = clientAuth,serverAuth
23 qcStatements = "ASN1:UTF8String:...statement PSP_AI PSP_PI PSP_CI..."
24
25 [ OIDs ]
26 organizationIdentifier = 2.5.4.97
```

6. Create Client CSR:

```
1 openssl req -config client_openssl.cnf -new -key client_private.key -nodes -out client.csr
```

7. Sign Client CSR by self signed CA

```
1 openssl x509 -req -days 360 -extfile client_openssl.cnf -extensions cert_ext -in client.csr
  -CAcreateserial -CA ca_certificate.crt -CAkey ca_private.key -out
  client_signed_certificate.crt
```

8. Check the certificate validity and its information

This can be done using Salt Edge TPP Verifier service, using this endpoint:

https://priora.saltedge.com/docs/tpp_verifier#certificates-verify-v2

For more information please refer to https://priora.saltedge.com/docs/tpp_verifier#certificates-verify